# COMP2400/6240 - Relational Databases

## Assignment 1 (SQL)

**Due date: 23:55, 3rd of April 2026**
**(Friday in Semester week 6)**

## Instructions

- **This assignment must be done individually.**

  Do not share your solutions, or partial solutions, with anyone. Do not post any idea/partial solution/result related to the assignment on the course discussion forum, or anywhere else where it can be read by other students.

  You may of course ask clarification questions, but make sure you phrase your questions so that they do not give away what you think the answer or solution is. If you have technical problems with accessing the `moviedb` database then you should ask for help. If you are not sure if a question is ok to post to the forum then ask your tutor during a lab or come to a drop-in session instead.

  Your answers to the assignment must be *your* answers. You can (and indeed should) use documentation, tutorials and other resources you can find on the web to learn SQL, and use what you learn to answer the assignment questions, but you should not obtain answers to the assignment questions from anywhere, anyone or anything. Your mark will be based on your original contribution to the answers only. Reference all resources (other than course material) that you have used in completing this assignment (by writing comments in your submitted SQL file).

- **You must submit your solutions through Canvas before the assignment deadline.** There will be an Assignment activity on the course Canvas page where you should upload your solution file. You can submit more than once, but we can only see, and will only mark, the last file that you submit.

  Late submissions will *not* be accepted. Submissions made outside the Assignment activity on Canvas (for example, files sent by email) will *not* be accepted, except in case of extensions approved after the extended submission due date. You will be marked on what you have submitted at the time of the deadline.

  If you find yourself in an unforeseeable situation beyond your control that you believe significantly affects your ability to complete the assessment on time, you can submit an application for extension or for ECA (`https://www.anu.edu.au/students/program-administration/assessments-exams/extenuating-circumstances-application`). Read the instructions on the college website (`https://systems.anu.edu.au/students/continuing/assessments/`) to find out in which case you should use one or the other, and how to apply. (Note, however, that ECA rules have recently changed, which is not yet reflected in those instructions: You should now apply for an ECA for any assessment, regardless of whether the weight is more or less than 20%, if you are seeking more than a 1 week extension or a different adjustment.)

- This assignment will count for 18% of the final course mark.

- A copy of the `moviedb` database is available in the PostgreSQL DBMS in the CSIT lab environment (including via `partch`). You can connect to the `moviedb` database by entering the following in a Terminal:

  ```
  psql moviedb
  ```

- (*Optional*) If you wish to set up your own copy of the movie database, you can use the `pg_dump` utility on a lab computer (or `partch`) to create an SQL file that will create and fill all tables in the database. You can then execute that script in your own database.

- You must submit one file: `u1234567.sql` containing your solutions to all the questions. For example, if your UID is u1234567, then name your submitted file as

u1234567.sql. **Please ensure your UID is correctly included in the file name (e.g., u1234567.sql). Submissions without a correct UID may not be identifiable and therefore may not receive a grade.** The file must be submitted through Canvas before the due date. You can download the template file from the folder "Assignment 1 (SQL)" on Canvas. You must enter your queries into the template file, and your submitted file `u1234567.sql` **must be executable, i.e., run successfully in PostgreSQL** (submissions that cannot be executed will receive a mark of zero) in the given database `moviedb`.

```
moviedb=> \i u1234567.sql
```

You can also test your queries against the `moviedb` database one by one following previous lab instructions.

Please write each of your answers below the comment indicating the corresponding question (`-- Q1` through `-- Q9`), and leave these comments unchanged. The file that you submit must have no more than one SQL query for each question, and must not contain any `psql` meta-commands.

- The correctness of queries should not depend on the database state, i.e., the content of the tables in the database. The current content in `moviedb` is provided to help you to get familiar with the database schema. To be fully correct, your SQL queries should produce correct answers when tested against other databases that have *the same schema, but different content*. Partial marks may be given if a query only has minor issues, but *any query that is not runnable* (i.e., that has syntax errors or is not correct w.r.t. the database schema) *will receive zero marks*.

  Note that we will test your submitted SQL queries against the PostgreSQL DBMS that is installed in the CSIT lab environment (currently, PostgreSQL version 15). Whether your queries are runnable or not will be determined by testing in this DBMS only.

- Please write your SQL queries in a clear, well formatted style, and use SQL comments (lines starting with `--`) to explain your thinking behind the crafting of a complex query. We may inspect queries that are not fully correct to determine if the flaw is minor. If your queries are well documented and easy to read and understand, this increases the chances that we will be able to find it partially correct.

- Sample SQL questions and solutions on `moviedb` will be available on Canvas, which will be helpful for you to work on your assignment.

- **Plagiarism, collusion, and undeclared use of disallowed tools will attract academic penalties in accordance with the ANU guidelines. Every student in this course is expected to be able to explain and defend any submitted assessment item. The course conveners can conduct or initiate an additional interview about any submitted assessment item for any student. If there is a significant discrepancy between the two forms of assessment, this will be seen as a case of potential academic misconduct.**

# The Movie Database

The schema of the relational database `moviedb` is shown in Figure 1.

There are five different categories of awards: movie awards, crew awards, director awards, writer awards and actor awards. The `result` attribute in the relations listing each type of award indicates if the movie/actor/director/writer/crew won the award, or was only nominated. Note that values of this attribute are not consistently capitalised (for example, both `'won'` and `'Won'` may appear). A movie that won an award was, of course, also nominated for that award, but the `result` attribute will only show that it won.

MOVIE(title, production_year, country, run_time, major_genre)
`primary key` : {title, production_year}

PERSON(id, first_name, last_name, year_born)
`primary key` : {id}

AWARD(award_name, institution, country)
`primary key` : {award_name}

RESTRICTION_CATEGORY(description, country)
`primary key` : {description, country}

DIRECTOR(id, title, production_year)
`primary key` : {title, production_year}
`foreign keys` : [title, production_year] ⊆ MOVIE[title, production_year]
[id] ⊆ PERSON[id]

WRITER(id, title, production_year, credits)
`primary key` : {id, title, production_year}
`foreign keys` : [title, production_year] ⊆ MOVIE[title, production_year]
[id] ⊆ PERSON[id]

CREW(id, title, production_year, contribution)
`primary key` : {id, title, production_year}
`foreign keys` : [title, production_year] ⊆ MOVIE[title, production_year]
[id] ⊆ PERSON[id]

SCENE(title, production_year, scene_no, description)
`primary key` : {title, production_year, scene_no}
`foreign keys` : [title, production_year] ⊆ MOVIE[title, production_year]

ROLE(id, title, production_year, description, credits)
`primary key` : {title, production_year, description}
`foreign keys` : [title, production_year] ⊆ MOVIE[title, production_year]
[id] ⊆ PERSON[id]

RESTRICTION(title, production_year, description, country)
`primary key` : {title, production_year, description, country}
`foreign keys` : [title, production_year] ⊆ MOVIE[title, production_year]
[description, country] ⊆ RESTRICTION_CATEGORY[description, country]

APPEARANCE(title, production_year, description, scene_no)
`primary key` : {title, production_year, description, scene_no}
`foreign keys` : [title, production_year, scene_no] ⊆ SCENE[title, production_year, scene_no]
[title, production_year, description] ⊆ ROLE[title, production_year, description]

MOVIE_AWARD(title, production_year, award_name, year_of_award, category, result)
`primary key` : {title, production_year, award_name, year_of_award, category}
`foreign keys` : [title, production_year] ⊆ MOVIE[title, production_year]
[award_name] ⊆ AWARD[award_name]

CREW_AWARD(id, title, production_year, award_name, year_of_award, category, result)
`primary key` : {id, title, production_year, award_name, year_of_award, category}
`foreign keys` : [id, title, production_year] ⊆ CREW[id, title, production_year]
[award_name] ⊆ AWARD[award_name]

DIRECTOR_AWARD(title, production_year, award_name, year_of_award, category, result)
`primary key` : {title, production_year, award_name, year_of_award, category}
`foreign keys` : [title, production_year] ⊆ DIRECTOR[title, production_year]
[award_name] ⊆ AWARD[award_name]

WRITER_AWARD(id, title, production_year, award_name, year_of_award, category, result)
`primary key` : {id, title, production_year, award_name, year_of_award, category}
`foreign keys` : [id, title, production_year] ⊆ WRITER[id, title, production_year]
[award_name] ⊆ AWARD[award_name]

ACTOR_AWARD(title, production_year, description, award_name, year_of_award, category, result)
`primary key` : {title, production_year, description, award_name, year_of_award, category}
`foreign keys` : [title, production_year, description] ⊆ ROLE[title, production_year, description]
[award_name] ⊆ AWARD[award_name]

Figure 1: The `moviedb` schema.

## Questions

Your task is to write SQL queries that answer the following questions. For each question, your answer must be a *single SQL query* (that may contain subqueries). Remember that your answers should not depend on the example database state.

Your queries must output any values from the database exactly as they are in the database; in particular, do not change upper/lower case of any query's output. (You may need to use case insensitive comparisons in some query conditions, but you should leave case unchanged in the output.)

You must write all your answers into the template file `u1234567.sql`.

1. List the number of crews that made a contribution to the music in a movie. List that number.
2. Count the number of movies that have restrictions from Chile, i.e., the restriction country is Chile. List that number.
3. List all movies' title and production_year that have never been nominated for an award. An award here can be movie/crew/director/writer/actor award. The query result should have two columns: the movie title, production year, and contains no duplicates.
4. We say a movie is a popular movie if it has been nominated for a movie award more than three times. Count the number of directors who have directed at least one popular movie. List that number. The term movie award here refers only to the awards listed in the MOVIE_AWARD table, not to those in other award tables (e.g., ACTOR_AWARD).
5. List all actors' first and last names who have appeared in at least six different scenes within one movie and the title and production year of that movie. The query result should have four columns, i.e., the actors' first and last names, and the movies' title and production year, and contains no duplicates.
6. List the ids of all directors who have directed at least one movie that has won at least one movie award and has fewer than three crews. The query result should have a single column, i.e., the directors' ids, and contain no duplicates. The term movie award here refers only to the awards listed in the MOVIE_AWARD table, not to those in other award tables (e.g., ACTOR_AWARD).
7. List the directors' first and last names who have directed at least one movie produced after year 1999 (NOT including 1999). The query result should have two columns: the first is directors' first name, the second is directors' last name, and no duplicates.
8. List the title and production year of all movies that were nominated for the actor award in the same year they were produced. The query results should have two columns: the movie title, production year, and no duplicates.
9. We say that two persons are co-crews if they have appeared as a crew in the same movie. List the first and last names of all crews that have the highest number of distinct co-crews. The query result should have two columns: the first column is crews' first names, the second column is crew's last names, and contains no duplicates.

**All done? Well done! Before you submit:**

- Check that your `u1234567.sql` file is *syntactically correct SQL*, by running it in `psql` connected to moviedb:

```
moviedb=> \i u1234567.sql
```

- Check that your solution (query) for each question is written below the comment line indicating the corresponding question ("`-- Qi`") in your `u1234567.sql` file, that your file has only one SQL query for each question, and that it does not contain any `psql` meta-commands.